

IMU SIGN LANGUAGE TRANSLATOR

SOFTWARE SETUP GUIDE

Hardware	Arduino UNO + 5× MPU6050 + TCA9548A + HC-05
Desktop Software	Python + Serial Monitor + Text-to-Speech
Python App	software/desktop_usb_voice/sign_language_usb_voice.py
Repository	Girdhari-Chaturvedi/Imu-Sign-Language-Translator

A practical Windows setup reference for running the desktop USB + voice application.

1. Software Architecture

The desktop side of the project reads serial telemetry from the Arduino UNO, displays detected gestures, and can speak detected SIGN messages using text-to-speech.

Layer	Software / Interface	Role
Firmware	Arduino IDE / .ino	Reads 5 MPU6050 sensors and sends DATA/SIGN messages
USB Link	Arduino USB Serial	Carries serial data to Windows
Desktop App	Python + Tkinter	Connects to COM port and displays data
Serial Library	pyserial	Reads Arduino serial messages
Voice	pyttsx3	Converts detected signs to speech

2. Recommended Windows Environment

The repository contains the desktop application and its requirements file. The setup below is intended for a Windows laptop/PC.

Step 1 — Install Python

Install a supported Python 3.x release and make sure Python is available from CMD. During installation, enable the option that adds Python to PATH.

Check:

```
python --version
```

If that command is unavailable, also try:

```
py --version
```

Step 2 — Open the Repository Folder

Open CMD or PowerShell and move to the downloaded/cloned repository:

```
cd Imu-Sign-Language-Translator
```

Step 3 — Create a Virtual Environment

A virtual environment keeps the project packages separate from other Python projects.

```
python -m venv venv
venv\Scripts\activate
```

After activation, the terminal normally shows **(venv)** at the beginning of the command line.

3. Install Python Dependencies

The desktop application's requirements file is located at:

```
software/desktop_usb_voice/requirements.txt
```

From the repository root, install the requirements with:

```
python -m pip install --upgrade pip
python -m pip install -r software\desktop_usb_voice\requirements.txt
```

Package	Purpose
pyserial	Serial communication with Arduino
pyttsx3	Offline text-to-speech
tkinter	Desktop GUI; normally included with standard Windows Python

4. Connect and Check the Arduino

Upload the project firmware from `firmware/arduino_5x_mpu6050/arduino_5x_mpu6050.ino` using Arduino IDE. Connect the Arduino UNO to the Windows PC using USB.

In Arduino IDE, select the correct board and COM port. Open Serial Monitor at 9600 baud to verify that the controller is producing DATA and SIGN messages.

Expected message examples:

```
DATA, S1, AX=0.120, AY=-0.940, AZ=0.310, GX=2.0, GY=-1.0, GZ=0.5, P=1.2, R=-70.3
SIGN:HELLO
```

5. Run the Desktop Application

With the virtual environment active, run:

```
python software\desktop_usb_voice\sign_language_usb_voice.py
```

The application should provide a COM-port selection, connection controls, detected-sign display, serial log, and text-to-speech for SIGN messages.

6. Typical Operating Flow

01	Connect Arduino USB	Arduino appears as a Windows COM port
02	Start desktop app	Select the Arduino COM port
03	Connect	App opens serial connection at 9600 baud
04	Move sensors	Firmware calculates motion features
05	Gesture detected	Arduino sends SIGN:<gesture>
06	Desktop response	GUI displays the sign and pyttsx3 speaks it

7. Troubleshooting

Problem	Check
python is not recognized	Check Python installation and PATH; try <code>py --version</code> .
ModuleNotFoundError	Activate venv and run the requirements installation command again.
COM port not visible	Reconnect USB, check Windows Device Manager, and close other serial programs.
COM port access denied	Close Arduino Serial Monitor or any other application using that COM port.
App connects but no data	Confirm firmware is uploaded and baud rate is 9600.
No voice output	Check Windows audio output and pytsx3 installation.
Incorrect gesture	Check sensor mounting, calibration, wiring, and prototype threshold rules.

8. Important Project Note

The current desktop application is a prototype interface. The firmware's gesture recognition uses threshold-based experimental mappings. It is not a complete or scientifically validated sign-language recognition system. For higher accuracy, collect labeled sensor time-series data and develop a trained classifier.

9. Repository Paths

Desktop application	software/desktop_usb_voice/sign_language_usb_voice.py
Python dependencies	software/desktop_usb_voice/requirements.txt
Arduino firmware	firmware/arduino_5x_mpu6050/arduino_5x_mpu6050.ino
Hardware documentation	hardware/
Project documentation	docs/

QUICK SETUP CHECKLIST

■	Python installed and visible in CMD
■	Repository opened in terminal
■	Virtual environment created and activated
■	requirements.txt installed
■	Arduino firmware uploaded
■	Arduino COM port identified
■	Arduino Serial Monitor tested at 9600 baud
■	Desktop Python application started
■	COM port connected in the application
■	SIGN messages displayed and spoken

End of Software Setup Guide